

Instructions

- Homework 2 was due Wednesday, July 29th at 18:00 Chicago Time.
 - All numbers and figures come from `homework_2_solution.ipynb`, run from top to bottom.
-

Problem 1: Hands-On OLS

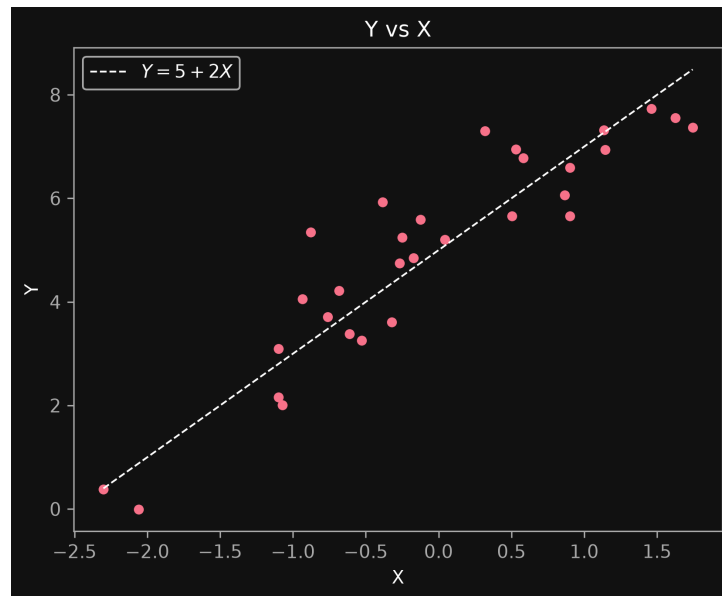
Problem 1.1: Setup

Set your random seed using `np.random.seed(1)`. Generate $n = 30$ observations where:

- The predictor X is drawn from a standard normal distribution, $X \sim N(0, 1)$.
- The error term ϵ is drawn from a normal distribution with mean 0 and standard deviation 1, $\epsilon \sim N(0, 1)$.
- The response variable is generated by the true relationship: $Y = 5 + 2X + \epsilon$.

Display a scatter plot of the generated data points (X_i, Y_i) .

Answer:



Problem 1.2: A First Fit

Using the data generated above, fit an OLS model. You should report:

1. $\hat{\beta}_0$ and $\hat{\beta}_1$ estimates.
2. Your confidence intervals and p-values for both coefficients.
3. The R^2 value of your fit.

Answer:

$R^2 = 0.840$ (adjusted 0.834).

	coef	std err	t	P> t	[0.025	0.975]
const	5.0677	0.155	32.787	0.000	4.751	5.384
X	1.8516	0.153	12.110	0.000	1.538	2.165

Problem 1.3: Interpretation

How do your estimated coefficients and confidence intervals compare to the true parameters?

Answer:

Close. $\hat{\beta}_0 = 5.0677$ against a true 5, and $\hat{\beta}_1 = 1.8516$ against a true 2, and both intervals cover the truth: $5 \in [4.751, 5.384]$ and $2 \in [1.538, 2.165]$. Both p-values round to 0, so both coefficients are significant. With $n = 30$ the intervals are still about ± 0.31 wide, so this is not a precise fit – only a correct one.

Problem 1.4: An Influential Point

Now, modify your dataset by overriding the last observation to be the point $(X_{30}, Y_{30}) = (4, -5)$. Refit your OLS model to this modified dataset, and report:

1. $\hat{\beta}_0$ and $\hat{\beta}_1$ estimates.
2. Your confidence intervals and p-values for both coefficients.
3. The R^2 value of your fit.

Answer:

$R^2 = 0.027$ (adjusted -0.008).

	coef	std err	t	P> t	[0.025	0.975]
const	4.5388	0.500	9.082	0.000	3.515	5.563
X	0.3533	0.402	0.879	0.387	-0.470	1.177

Problem 1.5: Interpretation

How do your estimated coefficients and confidence intervals compare to the true parameters?

Answer:

One point out of thirty destroys the fit. The slope falls from 1.8516 to 0.3533, its interval widens from ± 0.31 to ± 0.82 and now contains zero, the p-value rises from 0.000 to 0.387, and R^2 falls from 0.840 to 0.027.

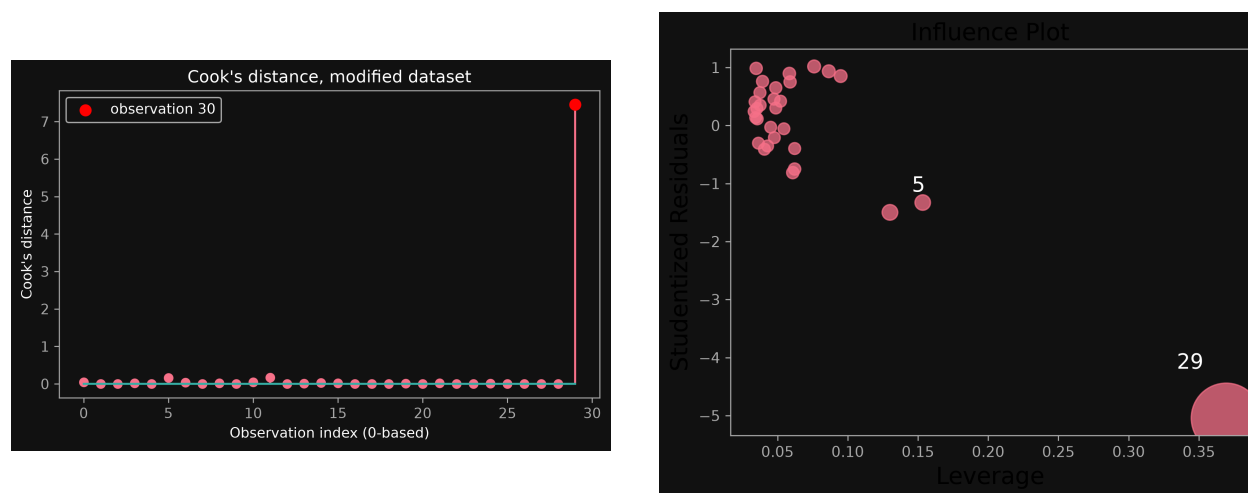
The true slope of 2 is now outside the interval, so the model has gone from correct and significant to wrong and insignificant.

Problem 1.6: Cook's Distance

Calculate Cook's distance for all observations in the modified dataset from Problem 1.4. Plot the Cook's distance values, and highlight the 30th observation.

Answer:

Cook's distance for observation 30 is 7.4530. The largest of the other twenty-nine is 0.1676, so it is more than forty times the next worst point.

**Problem 1.7: What if?**

Suppose instead that the influential point you just added was $(X_{30}, Y_{30}) = (0, -5)$.

What would you expect the *leverage* of this point to be relative to the $(4, -5)$ point you added before?

Answer:

Much lower. For a fit with an intercept,

$$h_{ii} = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_j (x_j - \bar{x})^2},$$

so leverage is driven by distance from $\bar{x}$, and $\bar{x} \approx 0$ here. Moving the point from $x = 4$ to $x = 0$ drops the second term to almost nothing and leaves h_{30} at the floor of $1/n = 0.0333$.

point	h_{30}	Cook's D_{30}	$\hat{\beta}_1$
$(4, -5)$	0.3695	7.4530	0.3533
$(0, -5)$	0.0335	0.4059	1.8077

The point is just as far off the line in both cases, but without leverage it cannot pull the slope: $\hat{\beta}_1$ stays at 1.8077 against the 1.8516 of the clean fit.

Problem 2: Central Limit Theorem

We found in Class 2, that if $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$, then as $n \rightarrow \infty$, then $\beta_{\text{OLS}} \rightarrow \mathcal{N}(\beta_{\text{OLS}}, \sigma_{\text{OLS}}^2)$, where $\sigma_{\text{OLS}}^2 = \sigma^2 / \sum_{i=1}^n (x_i - \bar{x})^2$.

The goal of this problem is for you to empirically verify that this results holds *even if* ϵ_i are not normally distributed.

Problem 2.1: Setup

First, define the true model as:

$$y = 2x$$

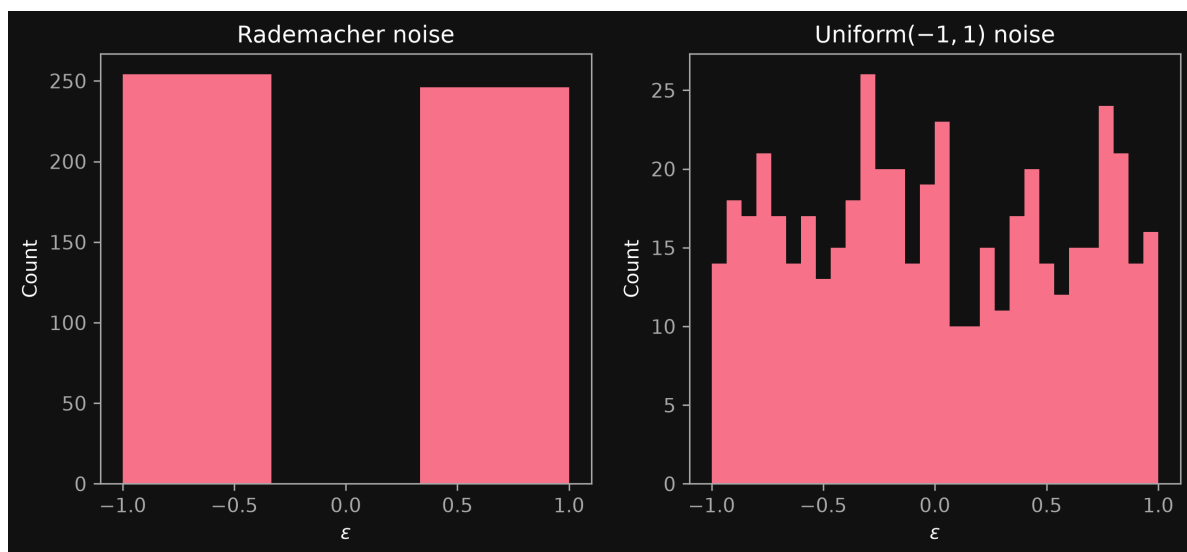
Where x is sampled from the uniform distribution $x \sim \text{Uniform}(0, 1)$. Note that we are not including an intercept, nor any noise.

Second, define the noise ϵ_i to be sampled from 2 different distributions:

- A Bernoulli distribution with $p = 0.5$, and values $\{-1, 1\}$ (the Rademacher distribution).
- A uniform distribution with $\epsilon_i \sim \text{Uniform}(-1, 1)$.

For $n = 500$, please display histograms of the 2 noise distributions you defined above.

Answer:

**Problem 2.2: Empirical Verification**

Note: You should not re-sample your x values, only the ϵ_i values. That is, you should have a fixed set of x values for this entire problem.

For $n \in 10, 100, 1000$, and for each noise distribution defined above, do the following:

1. Sample n values of ϵ_i from the noise distribution.
2. Generate the target values as $y_i = 2x_i + \epsilon_i$.
3. Fit an OLS regression to the data (x_i, y_i) , and obtain the estimate $\hat{\beta}_{\text{OLS}}$.
4. Repeat (1)-(3) 1,000 times, and save all of the $\hat{\beta}_{\text{OLS}}$ estimates.

You should now have 6 sets of 1,000 $\hat{\beta}_{OLS}$ estimates (2 noise distributions $\times$ 3 values of n).

Include your simulation code below:

Answer:

```

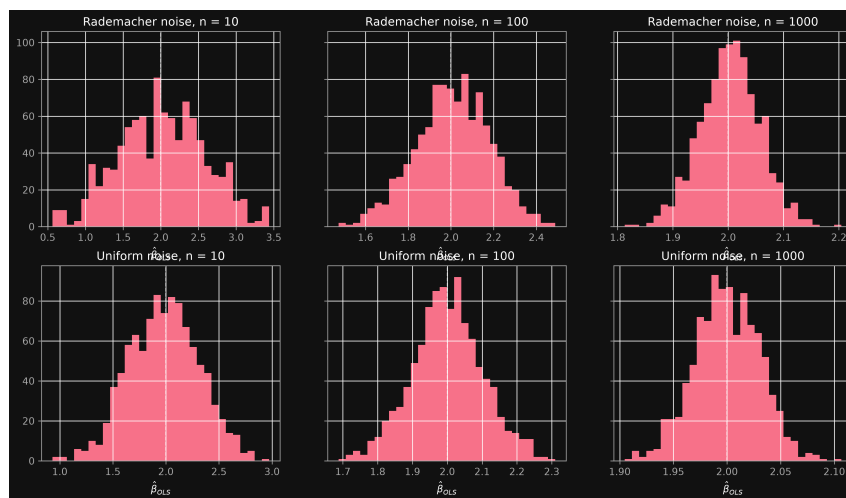
1 res_beta = {}
2 x = np.random.rand(1000) # fixed for the whole problem; only the noise is
  resampled
3 for n in [10, 100, 1000]:
4     x_n = x[:n]
5     draws = {"beta1": [], "beta2": []}
6     for _ in range(1000):
7         eps1 = 2 * np.random.binomial(1, 0.5, n) - 1
8         eps2 = 2 * np.random.rand(n) - 1
9         draws["beta1"].append(sm.OLS(2 * x_n + eps1, x_n).fit().params[-1])
10        draws["beta2"].append(sm.OLS(2 * x_n + eps2, x_n).fit().params[-1])
11    res_beta[n] = pd.DataFrame(draws)

```

Problem 2.3: Visualization

For each of the 6 sets of $\hat{\beta}_{OLS}$ estimates obtained above, plot a histogram of the estimates. Additionally, conduct a Shapiro-Wilk test for normality on each set of estimates, report your p-values, and briefly discuss your results.

Answer:



	10_beta1	10_beta2	100_beta1	100_beta2	1000_beta1	1000_beta2
shapiro_stat	0.9953	0.9983	0.9976	0.9983	0.9991	0.9988
shapiro_p	0.0038	0.4183	0.1475	0.4453	0.9025	0.7548

(beta1 is the Rademacher noise, beta2 the uniform.)

Only one cell rejects normality: Rademacher noise at $n = 10$, with $p = 0.0038$. Its p-value then

climbs to 0.1475 at $n = 100$ and 0.9025 at $n = 1000$, which is the central limit theorem doing its work on a noise term that is not remotely normal.

The uniform noise never rejects, not even at $n = 10$. An average of ten uniforms is already close to normal, whereas ten two-point draws are not, so the two distributions need different sample sizes before the same conclusion holds.

Problem 3: Weighted Least Squares

This problem focuses on verifying the findings of Class 2 regarding Weighted Least Squares (WLS).

Problem 3.1: Setup

Similar to before, define the true model as:

$$y = 2x$$

Where x is sampled from the uniform distribution $x \sim \text{Uniform}(1, 2)$.

Second, define the noise ϵ_i to be sampled from a normal distribution with mean 0, and variance σ_i^2 that depends on x_i as follows:

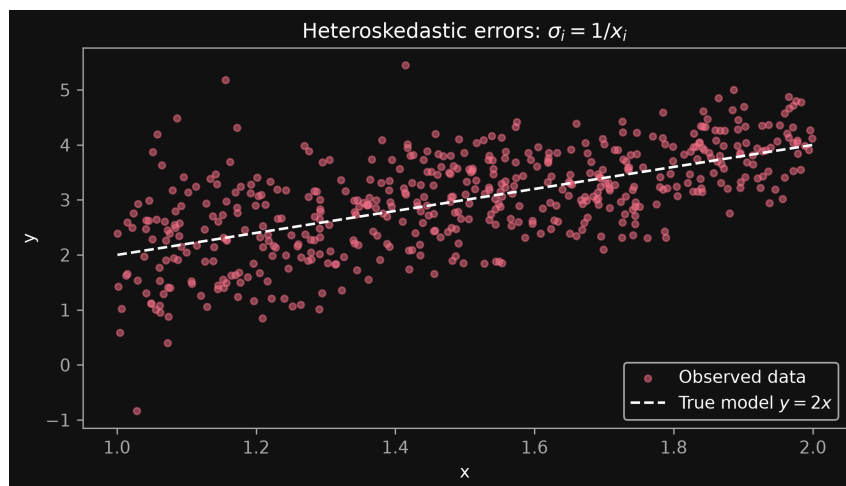
$$\sigma_i^2 = \frac{1}{x_i^2}$$

For $n = 500$, please plot the generated data points $(x_i, y_i + \epsilon_i)$.

Answer:

The true model has no intercept, so every fit below is through the origin, and the noise enters the response: $y_i = 2x_i + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, 1/x_i^2)$. That is what makes $w_i = x_i^2$ the correct weights in 3.5, since $w_i = 1/\sigma_i^2$.

```
1 np.random.seed(3)
2
3 n = 500
4 x = np.random.uniform(1, 2, n)
5 sigma = 1 / x
6 eps = np.random.normal(0, sigma)
7 y = 2 * x + eps
```



The vertical spread is widest at $x = 1$ ($\sigma_i = 1$) and narrowest at $x = 2$ ($\sigma_i = 1/2$), which is the heteroskedasticity we built in.

Problem 3.2: Naive OLS

Fit a standard OLS regression to the data generated above.

Report your estimate $\hat{\beta}_{OLS}$, and the confidence interval for the estimate.

Answer:

	coef	std err	t	P> t	[0.025	0.975]
x	2.0087	0.021	97.590	0.000	1.968	2.049

$\hat{\beta}_{OLS} = 2.0087$, with a 95% interval of $[1.9683, 2.0491]$. The point estimate is fine – OLS is still unbiased under heteroskedasticity. What is wrong is the standard error.

Problem 3.3: Interpretation

Briefly discuss whether naive OLS will be under or over confident in its estimate of $\hat{\beta}_{OLS}$, and why.

Answer:

Under-confident: the reported interval is wider than it needs to be.

For a fit through the origin, $\hat{\beta} - \beta = \sum_i x_i \epsilon_i / \sum_i x_i^2$, so the true variance is

$$\text{Var}(\hat{\beta}_{OLS}) = \frac{\sum_i x_i^2 \sigma_i^2}{(\sum_i x_i^2)^2} = \frac{\sum_i x_i^2 \cdot (1/x_i^2)}{(\sum_i x_i^2)^2} = \frac{n}{(\sum_i x_i^2)^2},$$

whereas OLS reports $\bar{\sigma}^2 / \sum_i x_i^2$, charging every point one common variance. The ratio of reported to true variance is

$$\frac{\bar{\sigma}^2 \sum_i x_i^2}{n} \longrightarrow \mathbb{E}\left[\frac{1}{x^2}\right] \mathbb{E}[x^2] = \frac{1}{2} \cdot \frac{7}{3} = \frac{7}{6},$$

so the reported standard error is about $\sqrt{7/6} \approx 1.08$ times too large.

In a fit through the origin the large- x points carry the most weight, and those are exactly the points with the *smallest* noise, since $\sigma_i = 1/x_i$. OLS charges them the average noise level instead, so it overstates its own uncertainty. The heteroskedasticity-robust (HC3) standard errors confirm it:

	$\hat{\beta}$	std err	CI lower	CI upper
OLS (conventional)	2.00870	0.02058	1.96826	2.04914
OLS (HC3 robust)	2.00870	0.01908	1.97131	2.04610

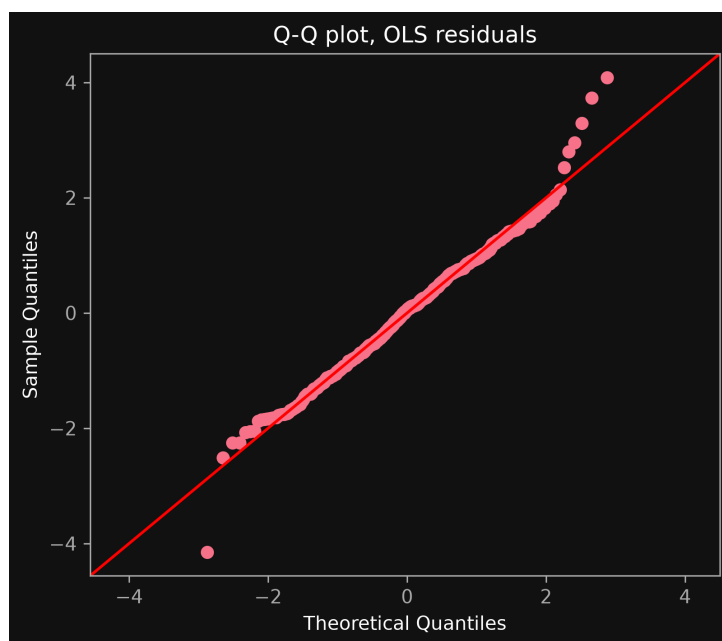
The realized ratio of the two standard errors is 1.0788, against the 1.0801 the algebra predicts in the limit.

Note that the direction is specific to this design. If the variance *grew* with x instead, the heavily weighted points would be the noisy ones and naive OLS would be over-confident.

Problem 3.4: Visualization

Plot a Q-Q plot of the residuals from the naive OLS regression. What do you observe?

Answer:



The points bend away from the 45° line in both tails. Each ϵ_i is normal on its own, but the residuals pool draws from 500 *different* variances – a scale mixture of normals, which is not itself normal. Shapiro-Wilk gives $p = 0.0011$.

Problem 3.5: Weighted Least Squares

Fit a Weighted Least Squares regression to the data generated above, using weights $w_i = x_i^2$. Report your estimate $\hat{\beta}_{\text{WLS}}$, and the confidence interval for the estimate.

Answer:

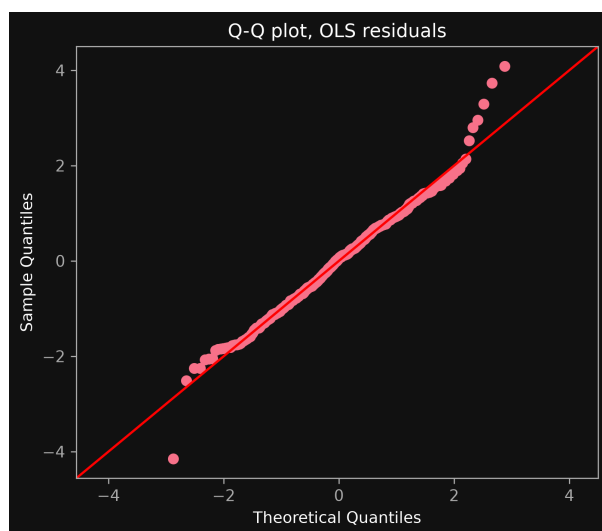
	coef	std err	t	P> t	[0.025	0.975]
x	2.0113	0.018	112.329	0.000	1.976	2.046

$\hat{\beta}_{\text{WLS}} = 2.0113$, with a 95% interval of $[1.976, 2.046]$. The weights $w_i = x_i^2 = 1/\sigma_i^2$ are exactly the GLS weights, so this is the efficient estimator: its standard error of 0.018 beats both the conventional and the robust OLS numbers from 3.3.

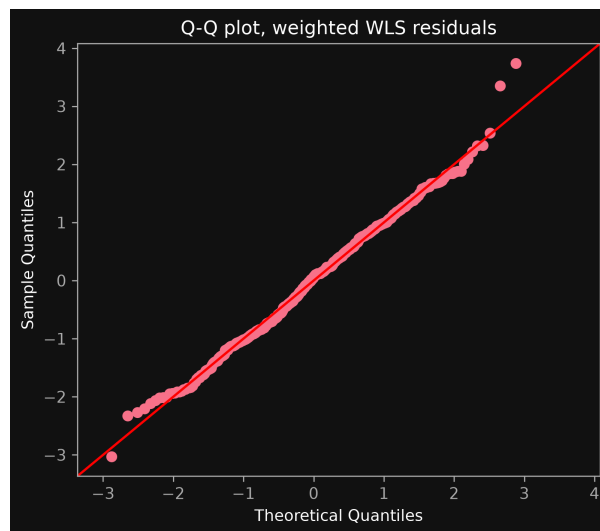
Problem 3.6: Visualization

Plot a Q-Q plot of the weighted residuals from the WLS regression. What do you observe compared to the Q-Q plot from the naive OLS regression?

Answer:



Naive OLS



WLS, weighted residuals

Much straighter, and the reason is exact: the weighted residual is $\sqrt{w_i} \epsilon_i = x_i \epsilon_i$, which has variance $x_i^2 \cdot (1/x_i^2) = 1$ for every i . Weighting turns the scale mixture back into a single standard normal.

	std dev	Shapiro p	corr($ \hat{\epsilon} , x$)
OLS residuals	0.6994	0.0011	-0.2577
Weighted (WLS) residuals	0.9852	0.1190	-0.0024

Note that `model_wls.resid` returns the *raw* residuals $y_i - x_i\hat{\beta}$, which are still heteroskedastic. The weighted ones are `model_wls.wresid`.